E-Commerce API Platform

3-Tier Architecture Testing & Validation Report

Date: 10 October 2026 | **Status:** 100% Passed | **Scope:** API-Led Connectivity

This document serves as the formal validation record for the E-Commerce API platform. Testing confirms the successful decoupling of the application into Experience, Process, and System layers, governed by a centralized Global Error Handling framework.

Synchronous & Intake Validation

Test Case 1: The Happy Path (End-to-End Fulfilment)

PASSED

Objective: Verify seamless communication across all layers, including database insertion, temporary file generation, and asynchronous SMTP email dispatch via LavinMQ.
Result: Payload successfully processed. The system returned a fast `202 Accepted` response while background processing completed the invoice dispatch.

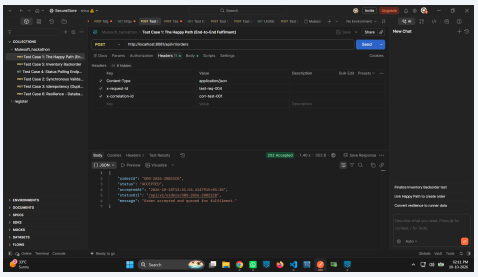


Fig 1: Postman capture showing 202 Accepted for valid order payload.

Test Case 2: Synchronous Validation (Bad Request)

PASSED

Objective: Prove the Experience Layer intercepts malformed business data (e.g., mismatched line totals) before allocating backend queue or database resources.
Result: DataWeave successfully blocked the payload, returning a precise `400 Bad Request`.

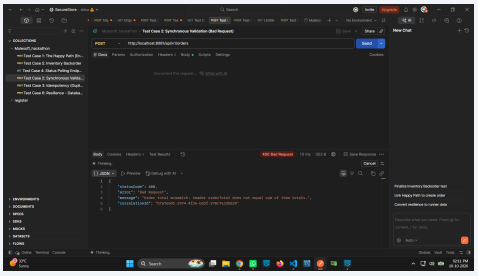


Fig 2: Postman capture showing 400 Bad Request for line total mismatch.

State Management & Idempotency

Test Case 3: Idempotency (Duplicate Request)

PASSED

Objective: Verify the Process Layer database check prevents duplicate order processing when identical request headers are submitted.
Result: System intercepted the duplicate `x-request-id` and safely returned the existing order details without duplicate database insertion.

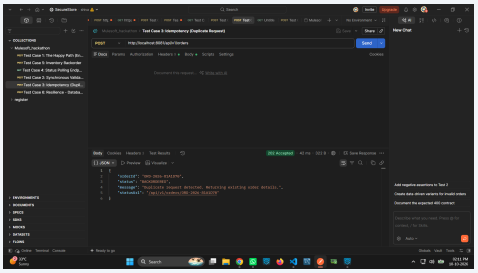


Fig 3: Postman capture showing Idempotency safety trigger.

Test Case 4: Status Polling Endpoint

PASSED

Objective: Validate the GET endpoint allows clients to query the System Layer for live, asynchronous order state.
Result: Returned `200 OK` with up-to-date line item status and fulfilment timestamps.

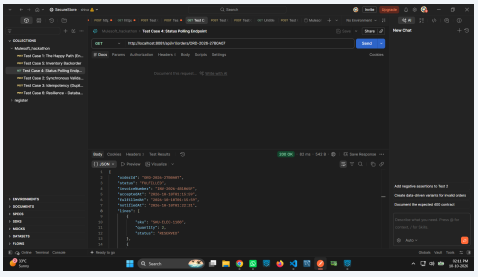
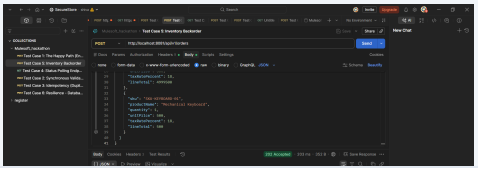


Fig 4: Postman capture showing successful order retrieval.

Test Case 5: Inventory Backorder Logic

PASSED

Objective: Prove the System Layer dynamically assesses inventory thresholds. Orders exceeding available stock should gracefully transition to a backordered state.
Result: High-quantity order correctly flagged as `BACKORDERED` in the database without halting the remaining pipeline.



Resilience & Error Handling

Test Case 6: Database Outage (503 Service Unavailable)

PASSED

Objective: Validate that the centralized Global Error Handler intercepts severed upstream connections and returns a secure JSON response, preventing stack trace leakage.
Result: Simulated database failure successfully mapped to `DB:CONNECTIVITY`, returning a clean 503 response.

Test Case 7: Async Poison Message (DLQ Routing)

PASSED

Objective: Prove fault tolerance. Malformed payloads injected into the queue must be caught and routed to a Dead Letter Queue (DLQ) to prevent infinite retry loops.
Result: Corrupted JSON was intercepted and parked safely in `order.fulfilment.queue.DLQ`.

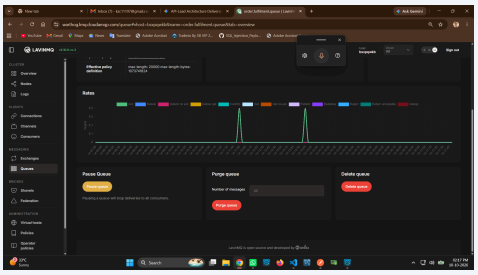


Fig 6: LavinMQ console capturing the active queues and DLQ architecture.

Application Log Verification

The following snippet from `testing.log` confirms successful backend asynchronous orchestration and DLQ routing:

Timestamp	Thread	Log Message Output
13:51:54	CPU_INTENSIVE	[INTAKE SUCCESS] Order ORD-2026-28B21CB accepted successfully.
14:01:15	BLOCKING	[IDEMPOTENT HIT] RequestId already exists: req-test-001
14:20:33	CPU_INTENSIVE	[FULFILMENT ERROR] Routing order to DLQ. Error: "Exception while reading 'Binary Content' as 'application/json'..."